OKS QUERY LANGUAGE

A Detailed Learning Guide for Understanding, Reading, and Constructing ATLAS TDAQ OKS Queries

Purpose. This document expands the material from the two supplied PDFs into a teaching-oriented reference. It is written to help a reader understand *why* each grammar construct exists, how to read a query from the outside inward, how nested expressions work, and how the constructs map to natural-language questions.

Important scope. The explanations and examples are grounded in the supplied documents. Where an example is illustrative rather than an official ATLAS example, it is explicitly described as illustrative. The supplied research also identified gaps such as the absence of official natural-language/query pairs and a Python OksQuery API.

Contents

1. The Mental Model: What Is an OKS Query?

The easiest way to understand OKS queries is to stop thinking of them as SQL statements and instead think of them as **instructions written as a tree**. An OKS query is an S-expression: a parenthesized expression whose first item identifies an operation and whose remaining items are that operation's arguments.

```
(operation argument1 argument2 ...)
```

The parentheses are not decoration. They tell the parser exactly which arguments belong to which operation. Because an argument can itself be another expression, an OKS query can contain expressions inside expressions.

For example, consider the documented query:

```
(all ("RunsOn" some (object-id "lxplus001.cern.ch" =)))
```

Do not try to understand it as one long string. Break it into layers:

```
all
■■■ ("RunsOn" some ...)
■■■ some
■■■ (object-id "lxplus001.cern.ch" =)
```

The outer **all** establishes the class scope. Inside it is a relationship condition for **RunsOn**. That relationship condition says **some** referenced object must satisfy the inner expression. The inner expression identifies the target object by its object ID.

Core principle: an OKS query is best understood from the outside inward. First determine the scope, then the logical condition, then any relationship traversal, and finally the innermost attribute/object test.

2. Reading an S-Expression as a Tree

Every opening parenthesis starts a new expression and every closing parenthesis ends it. The first token after an opening parenthesis normally tells you what kind of expression you are looking at.

Expression	What the first token tells you
(all ...)	Class scope including subclasses
(this ...)	Class scope excluding subclasses
(and ...)	All contained conditions must hold
(or ...)	At least one contained condition must hold
(not ...)	The contained condition must not hold
("Attribute" "Value" >)	Attribute comparison
(object-id "ID" =)	Object identity test
("Relationship" some ...)	Existential relationship test
("Relationship" all ...)	Universal relationship test
(path-to ...)	Relationship path traversal

A practical parsing technique is to repeatedly answer: *What is this expression asking the next level to do?*

Example:

```
(all (and ("Status" "Running" =) ("RunsOn" some (object-id "hostA" =))))
```

Read it as:

```
1. all → search the class and subclasses.
2. and → both conditions must hold.
3. First condition → Status must equal Running.
4. Second condition → RunsOn must have some related object.
5. object-id → that related object must be hostA.
```

This tree-based reading is also the most useful mental model for an NL-to-OKS system because the model must generate the same logical structure that the human question implies.

3. Top-Level Scope: all and this

The outermost query normally starts with either **all** or **this**. These are not Boolean operators. They define **which class hierarchy is searched**.

Grammar:

```
(all <expr>)
(this <expr>)
```

all means the specified class plus its subclasses. **this** means only the specified class. The supplied reference relates this distinction to the 'Search in subclasses' option in the OKS Data Editor.

Imagine a class hierarchy:

```
Application
■■■ RunControlApplication
■■■ MonitoringApplication
■■■ ControlApplication
```

If the current class is Application:

```
(all ("Status" "Running" =))
```

can match Application objects and objects belonging to its subclasses. By contrast:

```
(this ("Status" "Running" =))
```

restricts the search to Application itself.

Important: *all* has two different appearances in OKS grammar. At the top level, **all** means class scope. Inside a relationship expression, **all** is a quantifier meaning every referenced object. Context determines which role it plays.

4. Attribute Expressions

An attribute expression asks whether an object's attribute satisfies a comparison.

General form:

```
("attribute-name" "value" <comparator>)
```

The three conceptual parts are:

Part	Example	Question answered
Attribute name	"Max Speed"	Which property are we testing?
Value	"200"	What value are we comparing against?
Comparator	>	How should the comparison be performed?

So:

```
("Max Speed" "200" >)
```

means: **the object's Max Speed must be greater than 200**. The query does not itself say which class is being searched; the surrounding **all/this** expression provides the scope.

Another illustrative example:

```
("Manufacturer" "BMW" =)
```

This means the Manufacturer attribute must equal BMW. The exact attribute names available depend on the OKS schema/class being queried.

5. Comparators

The documented comparator set is:

```
= != ~= < > <= >=
```

Operator	Meaning	How to read it
=	Equal	attribute is exactly equal to the supplied value
!=	Not equal	attribute is different from the supplied value
~=	Regular-expression match	attribute matches a regular expression
<	Less than	attribute is numerically/order-wise less
>	Greater than	attribute is numerically/order-wise greater
<=	Less than or equal	attribute is not greater than the supplied value
>=	Greater than or equal	attribute is not less than the supplied value

Regular expression (~=). The supplied research states that ~= uses boost::regex and was added in release tdaq-01-09-00. This is different from equality: equality asks whether the values are the same, whereas a regular-expression comparison asks whether the value matches a pattern.

```
("Name" "^Run.*" ~=)
```

The preceding is an **illustrative** example of the syntax: it would conceptually ask whether Name matches the pattern ^Run.*. The supplied PDFs do not provide an official natural-language example for regex matching.

6. Object-ID Expressions

OKS objects have identifiers. An object-ID expression directly tests an object's identity.

```
(object-id "lxplus001.cern.ch" =)
```

Read this as: **is the current object the object whose ID is lxplus001.cern.ch?**

This construct becomes particularly useful when placed inside a relationship expression. For example:

```
("RunsOn" some (object-id "lxplus001.cern.ch" =))
```

Now the object-ID condition is not testing the application itself. It is testing an object reached through the **RunsOn** relationship.

The research report notes that object-ID lookup is more efficient than a non-indexed attribute search and can address objects that do not have non-key attributes. That makes object identity a distinct concept from ordinary attribute filtering.

7. Boolean Expressions

OKS provides three Boolean operators: **and**, **or**, and **not**. They allow simple conditions to be combined into more precise queries.

Grammar:

```
(and e1 e2 ...)
(or e1 e2 ...)
(not e)
```

and: every operand must be true. It requires at least two operands.

```
(and ("Status" "Running" =) ("Priority" "High" =))
```

Meaning: Status is Running **and** Priority is High.

or: at least one operand must be true. It requires at least two operands.

```
(or ("Manufacturer" "BMW" =) ("Manufacturer" "Audi" =))
```

Meaning: Manufacturer is BMW **or** Audi.

not: exactly one expression is supplied and its truth value is inverted.

```
(not ("Status" "Stopped" =))
```

Meaning: Status is not Stopped.

Operand-count rule: the supplied research explicitly notes that **and** and **or** require two or more operands, while **not** requires exactly one. This is a grammar rule, not merely a style preference.

8. Relationship Expressions

Relationships are what allow OKS queries to move from one object to another. Instead of asking only about an object's own attributes, a relationship expression asks about objects referenced by that object.

General grammar:

```
("relationship-name" some <expr>)
("relationship-name" all <expr>)
```

Suppose an Application has a relationship called **RunsOn**. That relationship can refer to Computer objects. The query can therefore ask whether one or more of those referenced Computer objects satisfy a condition.

```
("RunsOn" some (object-id "lxplus001.cern.ch" =))
```

The important observation is that the inner expression is evaluated against the **related object**, not the original object.

```
Application
|
| RunsOn
v
Computer
|
| object-id
v
lxplus001.cern.ch
```

This is why relationship expressions are essential for configuration databases: configuration information is represented as interconnected objects rather than only as rows of unrelated attributes.

9. some vs all — The Most Important Relationship Distinction

some is existential. It asks whether *at least one* referenced object satisfies the nested condition.

all is universal. It asks whether *every* referenced object satisfies the nested condition.

Consider an application with three RunsOn references:

```
RunsOn
■■■ hostA
■■■ hostB
■■■ hostC
```

Query A:

```
("RunsOn" some (object-id "hostB" =))
```

This is true because hostB is one of the referenced objects.

Query B:

```
("RunsOn" all (object-id "hostB" =))
```

This would require *every* referenced object to have the identity hostB. If the application also references hostA and hostC, the condition is false.

A useful natural-language mapping is:

English phrase	Likely logical idea
“runs on a host matching X”	some related host satisfies X
“has at least one related object matching X”	some
“every related object matches X”	all
“all referenced resources satisfy X”	all

The word **some** is therefore not simply a generic word. It has a precise quantifier meaning: *there exists at least one related object satisfying the expression*.

10. Combining and Nesting Expressions

The real power of the grammar comes from nesting. A Boolean operator can contain a relationship expression; the relationship expression can contain a Boolean expression; that Boolean expression can contain object-ID or attribute expressions.

Consider:

```
(all
(and
("Status" "Running" =)
("RunsOn" some
(or
(object-id "hostA" =)
(object-id "hostB" =)
)
)
)
)
```

Read it layer by layer:

```
Layer 1: all
Search the class and subclasses.

Layer 2: and
Both conditions must hold.
```

```
Layer 3, condition 1:
Status = Running.

Layer 3, condition 2:
RunsOn has some related object satisfying...

Layer 4:
The related object may be hostA OR hostB.

Layer 5:
object-id tests the identity of the related object.
```

Natural-language interpretation: **Find objects in the class hierarchy that are running and are associated through RunsOn with at least one of hostA or hostB.**

This illustrates why a flat keyword-matching system is insufficient. The system must preserve scope, Boolean structure, relationship direction, quantification and the location at which each condition is evaluated.

11. Path Queries: direct and nested

A normal relationship query is often used as a filter: 'does this object have a relationship to an object satisfying X?' A path query is different. It is designed to find a **sequence of linked objects** by navigating named relationships toward a destination object.

Documented top-level form:

```
(path-to "destination-object" <query-path-expression>)
```

The supplied grammar is:

```
query-path ::= '(path-to "destination-object" <query-path-expression>)'

query-path-expression ::=
'(<query-path-type> "rel-name"
[, "rel-name"]*
[<query-path-expression>])'

query-path-type ::= 'direct | nested'
```

direct and **nested** describe traversal modes. The query-path expression can itself contain another path expression, which is why path queries can represent multi-level configuration structures.

Documented pattern:

```
(path-to "my-id@my-class"
(direct "A" "B"
(nested "N"
(direct "X" "Y" "Z"))))
```

Conceptually, the traversal is:

```
Start object
■
■■■ A
■■■ B
■
▼
object reached through those relationships
■
■■■ N (nested)
■
▼
```

```
objects reached through N
■
■■■ X
■■■ Y
■■■ Z
■
▼
destination object
```

The supplied reference states that an unparseable path query produces **oks::bad_query_syntax**.

12. Real ATLAS Path Example

The research report contains a real **oks_dump** example:

```
(path-to "lxplus-3x3-21-ctrl@RunControlApplication"
(direct "Segments" "OnlineInfrastructure"
(nested "Segments"
(direct "Applications"
"IsControlledBy"
"Resources"))))
```

The important part is not memorizing this exact query. It is learning how to read it.

Piece	Role
path-to	Start a path query toward a destination object.
lxplus-3x3-21-ctrl@RunControlApplication	Destination object identifier.
direct Segments OnlineInfrastructure	Follow those relationships in direct traversal.
nested Segments	Enter the nested structure through Segments.
direct Applications IsControlledBy Resources	Continue through these relationships toward the destination.

The research describes this path in terms of Partition/Segment/OnlineInfrastructure/Applications/IsControlledBy/Resources relationships and records a result containing three objects along the path.

Key lesson: path queries describe the topology of the configuration graph. They are therefore different from ordinary attribute filters.

13. From English to an OKS Query

For an AI assistant, a useful approach is to translate the user's question into an intermediate logical representation before producing the final OKS syntax.

Question: “Which applications run on lxplus001.cern.ch?”

First identify the pieces:

Question element	OKS concept
Which applications?	Target class / class scope
run on	RunsOn relationship
lxplus001.cern.ch	Object ID of the related object
the host	Related object, so condition belongs inside RunsOn
Which applications	Return the outer objects satisfying the relationship

Then construct the tree:

```
all
■■■ RunsOn
■■■ some
■■■ object-id = lxplus001.cern.ch
```

Then serialize the tree into OKS grammar:

```
(all ("RunsOn" some (object-id "lxplus001.cern.ch" =)))
```

The supplied report identifies this as an official example. The key point is that the natural-language question is not converted by replacing words one-for-one. The system must determine *where* each piece of information belongs in the expression tree.

14. Programmatic C++ API

The research found C++ documentation for programmatically creating and executing OKS queries. The example creates an OksKernel, schema, class, attribute and object, then builds an OksQuery using an OksComparator.

```
OksKernel k;
k.new_schema("/tmp/car.schema");

OksClass *p =
new OksClass("Car", "Describes a car", false, &k);

OksAttribute *a = new OksAttribute("Max Speed", ...);
p->add(a);

OksObject *o = new OksObject(p, "BMW 316i");

OksQuery q(false,
new OksComparator(a,
new OksData((unsigned short)200),
OksQuery::greater_cmp));

OksObject::List *l = p->execute_query(&q);
```

Conceptually, the programmatic API separates the same ideas that appear in the string grammar: identify an attribute, construct a comparison, create a query object, execute it against a class, and receive matching objects.

The supplied research also notes that a query can be parsed from a string using the OksQuery string-query API and that **good()** can indicate whether the parsed query is valid. The research did not find a Python OksQuery API in the accessible documentation.

15. oks_dump: Executing Queries from the Command Line

oks_dump is useful both for interacting with OKS and, potentially, for validation/testing in an automated assistant.

```
oks_dump [--files-only]
[--class <name> [--query <query> [...]]]
[--path "<object-id>" "<path-query>"]
[--print-references]
[--print-referenced-by]
[--allow-duplicated-objects-via-inheritance]
[--version] [--help]
<database-file(s)>
```

--class identifies the class whose objects should be dumped; a query can be applied. **--path** executes a path query. Other options expose references, reverse references, version information and help.

A simple conceptual execution is:

```
oks_dump --class Application --query '(all ("RunsOn" some (object-id "hostA" )))'
database.data.xml
```

The exact database files, environment and command-line quoting depend on the OKS installation. The supplied research does not provide a complete deployment recipe, so this document should not be treated as a substitute for the installed OKS command help.

16. Errors and Validation

There are several different failure layers. Separating them helps when designing a query generator.

Error	What it indicates	Likely layer
oks::bad_query_syntax	The query/path syntax cannot be parsed.	Parsing / grammar
oks::BadReqExp	The regular expression used by ~= cannot be compiled.	Regex compilation
oks::QueryFailed	Query evaluation failed during execution.	Execution
oks_dump 1	Bad command line.	CLI parsing
oks_dump 2	Bad OKS file.	Input/configuration
oks_dump 3	Bad query.	Query validation/execution interface
oks_dump 4	No such class.	Schema/class lookup
oks_dump 5	Dangling references.	Configuration integrity
oks_dump 6	Exception caught.	Runtime exception

A useful validation pipeline is therefore:

```
Natural language
↓
Generate query
↓
Parse/validate grammar
↓
Check class/relationship names
↓
Execute
↓
Check execution result/error
↓
Answer user
```

The research gives practical repair guidance such as checking balanced parentheses, quoting, comparator tokens, command-line quoting, class names and dangling references, but explicitly labels these repair patterns as research-derived/inferred rather than official repair examples.

17. Schema, Classes, Attributes, Objects and Relationships

Understanding query grammar is only half the problem. A syntactically valid query can still be useless if it refers to a class, attribute or relationship that does not exist in the loaded schema.

The supplied research found complete DTD attribute lists for OKS schema/data formats. These describe properties such as type, range/format, initial value, nullability and ordering.

Relationship properties include concepts such as low/high cardinality and flags including composite, exclusive, dependent and ordered. These properties matter because they describe how objects are structurally connected.

The research found only a partial concrete class/object inventory because large files such as **all_types.schema.xml**, **test.data.xml** and **DAQ-Configuration.schema.xml** were truncated during fetching.

Verified example class names in the collected research include:

```
Car
Computer
Partition
Segment
RunControlApplication
```

These names should not be treated as an exhaustive ATLAS inventory. The research explicitly identifies the inventory as incomplete.

18. Run / Revision Mapping

Run/revision mapping is part of the broader configuration-service research rather than the core query grammar. The collected research identified several related mechanisms and identifiers.

The tag format documented in the research is:

```
tag:<run-number>@<partition>
```

The research also identifies TDAQ_DB_VERSION values, the RunNumber database schema/class and rn_ls usage as relevant to mapping a run to a configuration/revision. It additionally cites a Run 3 configuration-service paper.

The important conceptual distinction is:

```
Query grammar
→ describes how to select/navigate configuration objects.

Run/revision mapping
→ helps determine which configuration state/database/revision
corresponds to a particular run or partition.
```

Because the supplied material does not provide a complete end-to-end run-to-revision implementation recipe, this section should be treated as a research overview rather than an operational procedure.

19. What This Means for an MCP / AI Assistant

The two PDFs suggest that the core AI problem is best treated as **structured query generation**, not merely document retrieval.

A robust conceptual architecture is:

```
User
↓
Natural-language understanding
↓
Identify class / attributes / relationships / IDs / time context
↓
Build an intermediate query tree
↓
Serialize into OksQuery grammar
↓
Validate with authoritative parser or OKS tooling
↓
Execute read-only query
```

```
↓
Interpret returned objects
↓
Natural-language response
```

Why an intermediate tree helps: it allows the system to inspect whether the generated structure makes sense before producing the final string. For example, it can verify that a relationship contains a relationship expression, that **not** has one operand, that **and/or** have enough operands, and that an object-ID test appears where an object condition is expected.

Training-data problem: the supplied research found zero official natural-language-to-OKS-query pairs and zero official invalid-query examples. Therefore, a future dataset would need to generate valid grammar-constrained queries, validate them against the actual parser, generate natural-language paraphrases, and create machine-checked negative examples.

Validation recommendation from the research: port/wrap the C++ OksQuery parser or use **oks_dump** as a validation/execution oracle on a system where OKS is installed.

20. Practice Exercises

These exercises are designed to test whether you can reason about the grammar rather than memorize strings. Some use generic names because the supplied PDFs do not provide a complete real-world class/attribute inventory.

Exercise 1. What does this mean?

```
(this ("Status" "Running" =))
```

Answer: search only the specified class, not its subclasses, for objects whose Status equals Running.

Exercise 2. What is the difference between these?

```
(all ("RunsOn" some (object-id "hostA" =)))
(all ("RunsOn" all (object-id "hostA" =)))
```

Answer: the first requires at least one RunsOn target to be hostA. The second requires every RunsOn target to be hostA.

Exercise 3. What is wrong with this expression?

```
(not ("A" 1) ("B" 2))
```

Answer: not accepts exactly one expression. The expression contains two operands.

Exercise 4. What is wrong with this expression?

```
(and ("A" "1" =))
```

Answer: the supplied grammar requires and to have at least two operands.

Exercise 5. Construct the logical tree for: “running AND on hostA OR hostB.”

Answer: an outer AND contains Status=Running and a RunsOn/SOME expression whose inner condition is OR(object-id=hostA, object-id=hostB).

Exercise 6. What is the role of object-id inside a relationship expression?

Answer: it tests the identity of the related object reached through that relationship, rather than testing the original object's identity.

Exercise 7. What is a path query for?

Answer: it describes navigation through named relationships to find a path toward a destination object; it is different from a simple attribute filter.

21. Final Grammar Cheat Sheet

Top-level scope

```
(all <expr>)
(this <expr>)
```

Boolean

```
(and e1 e2 ...)
(or e1 e2 ...)
(not e)
```

Attribute

```
("attribute-name" "value" <comparator>)
```

Object identity

```
(object-id "object-id" =)
```

Relationship

```
("relationship-name" some <expr>)
("relationship-name" all <expr>)
```

Comparators

```
= != ~= < > <= >=
```

Path

```
(path-to "destination-object" <query-path-expression>)
```

Path modes

```
direct
nested
```

Operand constraints

```
and → 2 or more
or → 2 or more
not → exactly 1
```

Best way to read any query

```
1. Identify all/this.
2. Find the main expression inside it.
3. If it is and/or/not, determine the Boolean structure.
4. If it is a relationship, identify the relationship name and some/all.
5. Move into the nested expression.
6. Resolve attribute or object-id comparison.
7. Check every parenthesis and quote.
8. Check that names actually exist in the schema.
```

Best way to generate a query programmatically

```
Natural language
→ structured intent
→ query tree
→ grammar serialization
→ parser validation
```

→ OKS execution
→ result interpretation

This final workflow captures the central lesson of the supplied research: OKS query generation is a structured-language problem, and correctness depends on both grammar validity and knowledge of the actual OKS schema/configuration.

Source & Limitations

This learning guide is an expanded explanation of the two user-supplied documents: the ATLAS TDAQ OKS research report and the OKS Query Language reference. It does not claim that the illustrative examples are official unless the source material identified them as such.

The supplied research explicitly reports several limitations: no official natural-language/query pairs; no official invalid-query examples; no Python OksQuery API found in the accessible material; incomplete concrete inventories because large files were truncated; and authenticated CERN documentation that could not be accessed anonymously.

Accordingly, this guide should be used as a conceptual and grammar-learning reference. For production query generation, the actual OKS installation, schema files, parser and command-line tools should remain the authoritative source for the current environment.